ELEC-E8126 - Robotic manipulation

Student Number: 802651
Name: Tzu-Lin, Huang
Exercise No. 2

This report using Euclidean Distance as performance metric

PRM

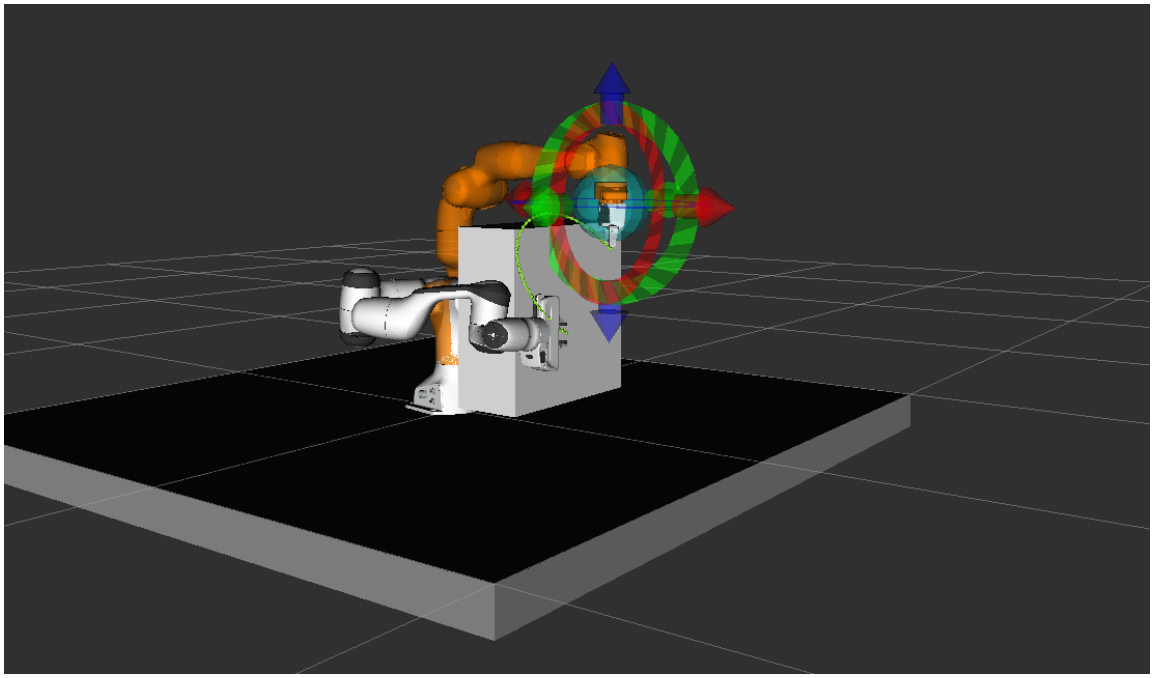
Created 1628 states
Solution found in 5.004529 seconds
Number of Trajectory Points :40
Trajectory Path Length (Euclidean Distance) 0.672768

Created 1577 states
Solution found in 5.004361 seconds
Number of Trajectory Points :40
Trajectory Path Length (Euclidean Distance) 0.672789

Created 1661 states
Solution found in 5.005729 seconds
Number of Trajectory Points :40
Trajectory Path Length (Euclidean Distance) 0.672696

Created 1657 states
Solution found in 5.002703 seconds
Number of Trajectory Points :40
Trajectory Path Length (Euclidean Distance) 0.672861

Created 1566 states
Solution found in 5.004468 second
Number of Trajectory Points :40
Trajectory Path Length (Euclidean Distance) 0.672859



RRT

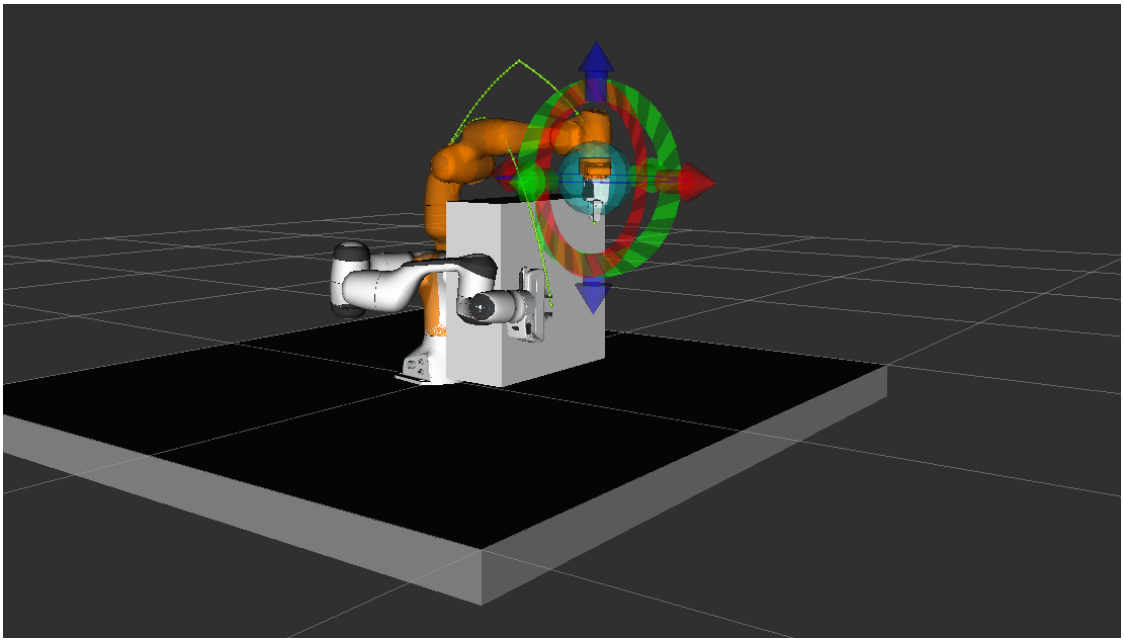
Created 23 states
Solution found in 0.025304 seconds
Number of Trajectory Points :174
Trajectory Path Length (Euclidean Distance) 4.53895

Created 19 states
Solution found in 0.023703 seconds
Number of Trajectory Points :75
Trajectory Path Length (Euclidean Distance) 1.83587

Created 25 states
Solution found in 0.021231 seconds
Number of Trajectory Points :93
Trajectory Path Length (Euclidean Distance) 4.35529

Created 639 states
Solution found in 0.758302 seconds
Number of Trajectory Points :266
Trajectory Path Length (Euclidean Distance) 5.46839

Created 20 states
Solution found in 0.033700 seconds
Number of Trajectory Points :108
Trajectory Path Length (Euclidean Distance) 4.20848



RRTstar

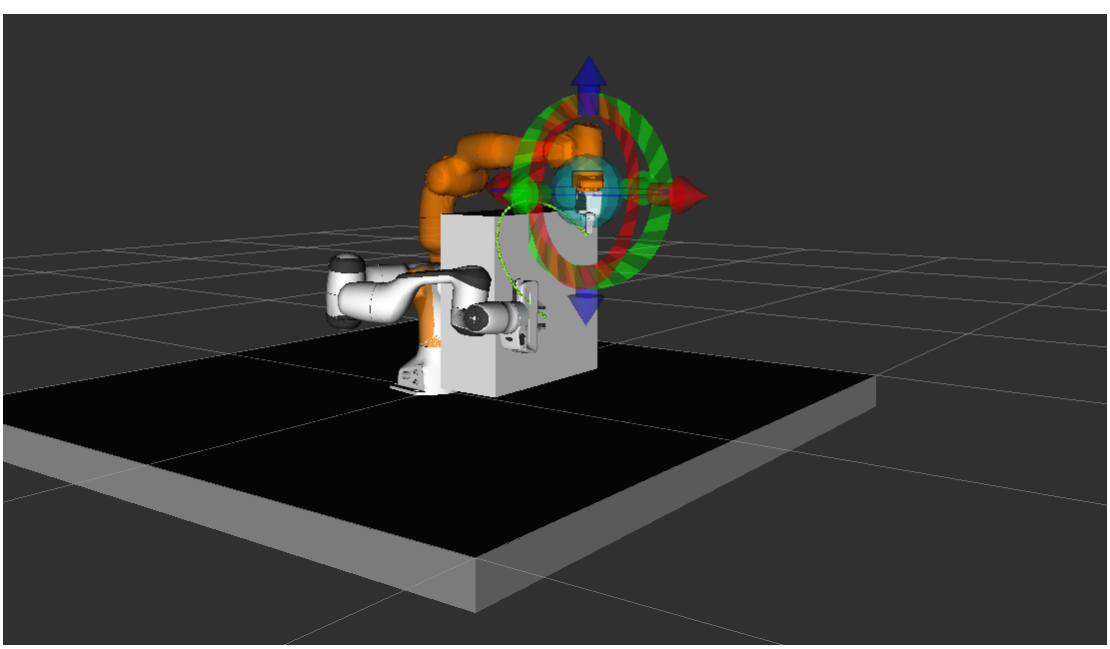
Found an initial solution with a cost of 6.52 in 61 iterations (28 vertices in the graph)
Solution found in 5.000628 seconds
Number of Trajectory Points :40
Trajectory Path Length (Euclidean Distance) 0.672685

Found an initial solution with a cost of 6.52 in 30 iterations (20 vertices in the graph)
Solution found in 5.000633 seconds
Number of Trajectory Points :40
Trajectory Path Length (Euclidean Distance) 0.672845

Found an initial solution with a cost of 6.52 in 35 iterations (16 vertices in the graph)
Solution found in 5.001824 seconds
Number of Trajectory Points :40
Trajectory Path Length (Euclidean Distance) 0.672884

Found an initial solution with a cost of 6.52 in 23 iterations (10 vertices in the graph)
Solution found in 5.001118 seconds
Number of Trajectory Points :40
Trajectory Path Length (Euclidean Distance) 0.672797

Found an initial solution with a cost of 6.52 in 40 iterations (25 vertices in the graph)
Solution found in 5.000229 seconds
Number of Trajectory Points :41
Trajectory Path Length (Euclidean Distance) 1.0587



KPIECE

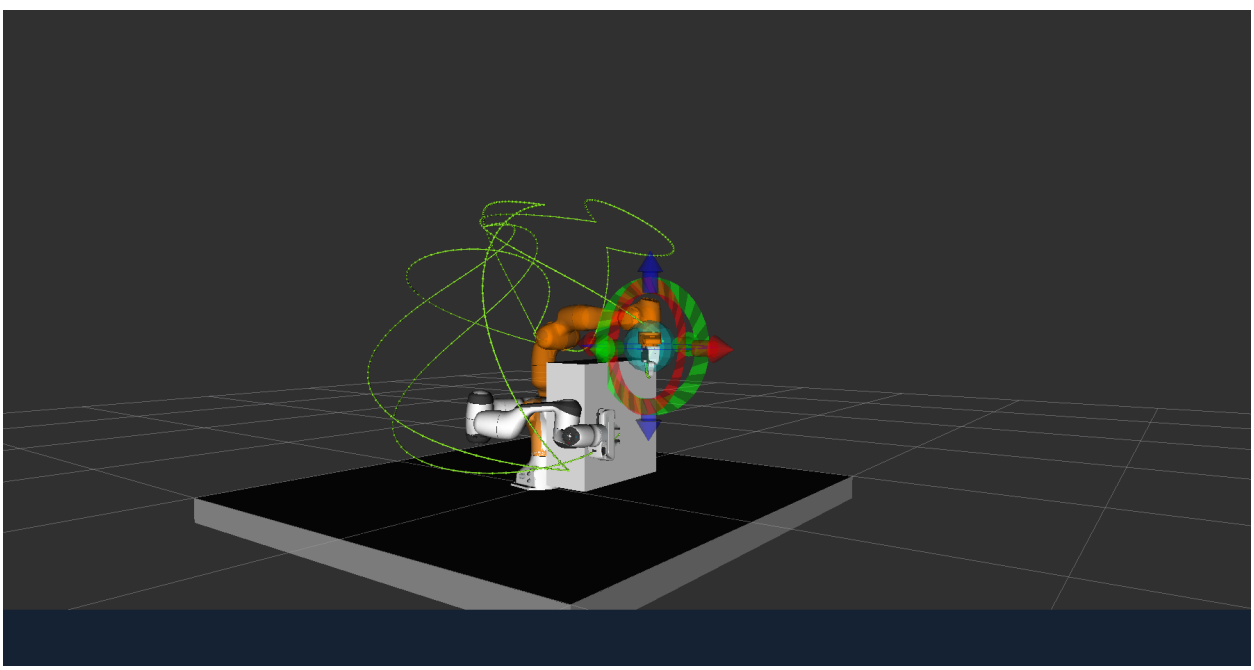
Created 10 states in 10 cells (0 internal + 10 external)
Solution found in 0.012746 seconds
Number of Trajectory Points :98
Trajectory Path Length (Euclidean Distance) 3.5618

Created 8 states in 8 cells (0 internal + 8 external)
Solution found in 0.010143 seconds
Number of Trajectory Points :40
Trajectory Path Length (Euclidean Distance) 0.67282

Created 12 states in 12 cells (0 internal + 12 external)
Solution found in 0.015008 seconds
Number of Trajectory Points :141
Trajectory Path Length (Euclidean Distance) 4.35922

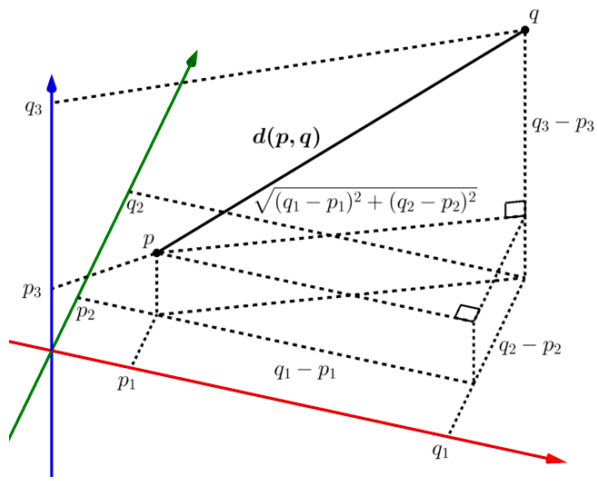
Created 14 states in 14 cells (0 internal + 14 external)
Solution found in 0.021403 seconds
Number of Trajectory Points :211
Trajectory Path Length (Euclidean Distance) 6.93364

Created 32 states in 32 cells (0 internal + 32 external)
Solution found in 0.054373 seconds
Number of Trajectory Points :492
Trajectory Path Length (Euclidean Distance) 15.2975



This report using Euclidean Distance as performance metric

1. mathematical equation of plan length computation



In Cartesian coordinates, if **p** = (p_1, p_2,..., p_n) and **q** = (q_1, q_2,..., q_n) are two points in Euclidean *n*-space, then the distance (d) from **p** to **q**, or from **q** to **p** is given by the Pythagorean formula: (source: https://en.wikipedia.org/wiki/Euclidean_distance)

$$d(\mathbf{p},\mathbf{q}) = d(\mathbf{q},\mathbf{p}) = \sqrt{(q_1-p_1)^2+(q_2-p_2)^2+\cdots+(q_n-p_n)^2}$$

$$= \sqrt{\sum_{i=1}^{n}(q_i-p_i)^2}.$$

In three-dimensional Euclidean space, the distance is

$$d(\mathbf{p},\mathbf{q}) = \sqrt{(p_1-q_1)^2+(p_2-q_2)^2+(p_3-q_3)^2}.$$

2. discussion of the results (why plan X performed better than Y w.r.t. metric Z)

PRM

The PRM algorithm use probabilistic sampling to create a graph of the free configuration space for multiple-query planning. RRT is faster than PRM due to the number of calculated edges are lower, and instead of exploring the c space every time to plan a path like RRT does, PRM explore multiple times from generated graph as long as the environment does not change.

RRT V.S. RRT* (RRTstar)

The RRT algorithm build a collision-free graph, which points are randomly generated and closest node is selected to connect.

RRT* (RRTstar) is a modified version of RRT by achieving optimal solution for shortest path. It is done by rewiring the tree if a shorter path between 2 points is available, and the outcome trajectories is smoother than the ones obtained with RRT algorithm.

KPIECE

The KPIECE algorithm try to use the minimum time to find a solution, which mainly design for optimal time not for optimal path. When we compared the result with RRT or RRT* (RRTstar), there are huge difference between planning (solution) time and euclidean distance metric.

3. answers to the following questions:

- **–** Assuming infinite planning time, will there be a difference in planned path length of RRT and RRT* in MoveIt?

The RRT algorithm may continue to improve as time goes by, however, the RRT dose not converge to an optimal solution. As for RRT* (RRTstar) continue rewires the search tree to ensure that it always encodes the shortest path, so at the end will approach an optimal solution through number of sample nodes increase.

- In the evaluation you should compute 5 plans per planner. Why is it necessary to compute more plans with the same planner to get meaningful comparison?

Short answer will be most of the planners based on random tree algorithm or sampling algorithm, and it is crucial to execute multiple times to do benchmark.

Reference

[1] Noreen, Iram, A. R. Khan and Zulfiqar Habib. (2016) "A Comparison of RRT , RRT * and RRT *-Smart Path Planning Algorithms”.

[2] MODERN ROBOTICS MECHANICS, PLANNING, AND CONTROL